

Daily Organizer — Complete Documentation

Written for a frontend developer (HTML/CSS/JS + a little Angular) who wants to understand how the full project works — frontend, backend, database, and everything in between. No prior backend/API experience assumed.

Table of Contents

- [1. What is this project?](#)
- [2. The big picture: how a modern web app works](#)
- [3. The three layers explained](#)
- [4. Tech stack & why each piece exists](#)
- [5. Project structure \(annotated\)](#)
- [6. Following a click: full data flow example](#)
- [7. Authentication explained](#)
- [8. Feature deep-dives](#)
 - [My Plans \(Tasks\)](#)
 - [Goals](#)
 - [Daily Routine \(Habits\)](#)
 - [Calendar](#)
 - [Dashboard](#)
- [9. Recurring patterns you'll see in the code](#)
- [10. Adding a new feature module](#)
- [11. Running the project locally](#)
- [12. Glossary](#)

1. What is this project?

Daily Organizer is a personal productivity web app. One user logs in and can:

- Plan tasks, trips, meetings, dinners, reminders (collectively called "plans")
- Set goals broken down into milestones and mini-goals
- Build daily habits with streaks and a 30-day heatmap (Daily Routine)
- View everything on a calendar
- Connect a Google Calendar to push plans into it

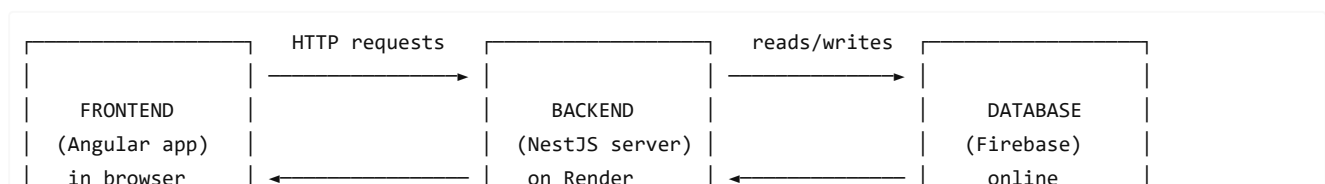
It's a **full-stack** app: there's a **frontend** (what you see in the browser), a **backend** (a separate program running on a server), and a **database** (where data is stored permanently). All three are in this repo:

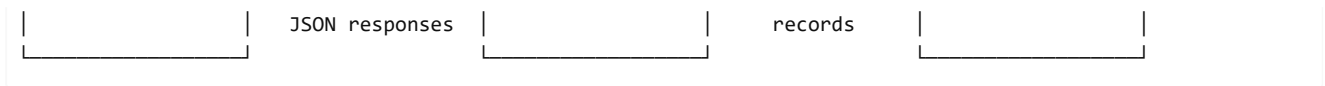
```
Daily Organiser/  
├── frontend/    ← Angular app (what runs in your browser)  
├── backend/     ← NestJS API server (runs on Node.js)  
└── README.md   ← project entry doc
```

2. The big picture: how a modern web app works

If you've only built static HTML/CSS/JS sites, the jump to "full-stack" can feel mysterious. Here's the mental model.

Three independent programs that talk to each other





Each box is a **separate program**:

- **Frontend** = the HTML/CSS/JS files Angular generates. Lives in the user's browser. Has no direct access to the database. Can only ask the backend for data via HTTP.
- **Backend** = a Node.js program that runs on a server somewhere (locally for dev, Render in production). It listens for HTTP requests, validates them, reads/writes the database, and sends back JSON responses.
- **Database** = a remote system that just stores data. Frontend never talks to it directly — only the backend has the credentials.

The frontend and backend talk over **HTTP** (the same protocol your browser uses to load any web page). The backend talks to Firebase using Firebase's own SDK.

Why three layers instead of one?

Security. If the frontend talked to the database directly, the database password would be in the browser — anyone could steal it. The backend keeps the secrets and acts as a gatekeeper.

Flexibility. You could swap the frontend for a mobile app, or swap the database for PostgreSQL, without rewriting the other layers. The HTTP contract stays the same.

Logic. Things like "only this user can edit this task" need to be enforced somewhere trusted. The browser can't be trusted (users can modify it), so the backend is where rules live.

3. The three layers explained

HTTP: the language between frontend & backend

HTTP is a simple request/response protocol. Every interaction is:

1. The frontend sends a **request** (a verb + URL + optional body).
2. The backend sends back a **response** (a status code + body).

Common verbs:

Verb	Meaning	Example
GET	"give me this data"	GET /api/tasks → list all tasks
POST	"create something new"	POST /api/tasks with body {title: "Buy milk"}
PATCH	"update part of something"	PATCH /api/tasks/abc123 with body {status: "DONE"}
DELETE	"remove something"	DELETE /api/tasks/abc123

Common status codes:

- 200 OK — request succeeded
- 201 Created — request succeeded and made something new
- 400 Bad Request — your request body was wrong
- 401 Unauthorized — you're not logged in or your token is invalid
- 403 Forbidden — you're logged in but not allowed to do this
- 404 Not Found — the thing you asked for doesn't exist
- 500 Server Error — backend bug

The body of requests and responses is **JSON**:

```
{ "id": "abc", "title": "Buy milk", "status": "DONE" }
```

JSON is just a text format that looks like a JavaScript object. Both frontend and backend can parse/serialize it.

API: a contract between frontend & backend

"API" is just the **list of HTTP endpoints the backend exposes** and what each one does. For example:

- `GET /api/habits` returns an array of habits
- `POST /api/habits` accepts a body like `{title, weekdays, ...}` and creates one

In this project, the API base URL is `http://localhost:3000/api` in dev (see `backend/src/main.ts`).

Database: where data actually lives

This project uses **Firebase Realtime Database** — a hosted, JSON-tree database from Google. It's not SQL; it's just a big tree of JSON. The shape looks roughly like:

```
firebase-root/
├─ users/
│   └─ <userId>/ { email, username, passwordHash, ... }
├─ tasks/
│   └─ <taskId>/ { title, userId, status, ... }
├─ goals/
│   └─ <goalId>/ { title, userId, progress, ... }
├─ milestones/
│   └─ <milestoneId>/ { goalId, status, ... }
├─ habits/
│   └─ <habitId>/ { title, weekdays, userId, ... }
├─ habitCheckins/
│   └─ <checkinId>/ { habitId, userId, date, ... }
└─ categories/
    └─ <categoryId>/ { name, color, userId }
```

Every record has its own unique ID. Records reference each other by ID (e.g., a task has a `userId` field pointing to which user owns it).

The backend uses Firebase's Admin SDK to read/write this tree. See `backend/src/prisma/firebase.service.ts` — that's the wrapper that exposes `get`, `set`, `update`, `remove`, `getList` methods used by every backend service.

Why is the file called *prisma*? Historical. The project originally used Prisma + PostgreSQL. When it migrated to Firebase, the folder and module names stayed for compatibility. The actual implementation is purely Firebase.

4. Tech stack & why each piece exists

Frontend: Angular 19

Angular is a JavaScript framework that helps build big single-page apps (SPAs). Why not just HTML/CSS/JS?

Problem	Angular's answer
Sharing logic across many pages without copy-paste	Components — reusable building blocks (<code><app-sidebar></code> , <code><app-modal></code>)
Updating the DOM when data changes	Data binding — <code>{{ variable }}</code> in the template updates automatically when variable changes
Talking to a backend	HttpClient service — handles fetch/axios-style calls cleanly
Routing without page reloads	Router — <code>/dashboard</code> VS <code>/tasks</code> switches without refresh
Shared state between unrelated components	Services + RxJS — a singleton class others inject

Specifically this project uses **standalone components** (the newer Angular 19 style — no NgModule boilerplate) and **signals** (`signal(...)` , `computed(...)`) for reactive state.

Backend: NestJS 11

NestJS is a Node.js framework for building backend APIs. It's heavily inspired by Angular — same idea of decorators, dependency injection, modules.

Problem	NestJS answer
Defining HTTP endpoints	Controllers with <code>@Get()</code> , <code>@Post()</code> decorators on methods
Reusing business logic across controllers	Services — classes with <code>@Injectable()</code> , automatically wired up
Validating incoming request bodies	DTOs + class-validator decorators (<code>@IsString()</code> , <code>@IsOptional()</code>)
Organizing related code	Modules — <code>TasksModule</code> bundles <code>TasksController</code> + <code>TasksService</code>
Auth & permissions	Guards — run before a controller method, can deny access

A typical request flow inside NestJS:

1. HTTP request arrives at a `@Controller('xyz')` -decorated class
2. Global `JwtAuthGuard` checks for a valid JWT in the `Authorization` header
3. `ValidationPipe` validates the body against a DTO
4. The matching method runs (e.g., `findAll(userId)`)
5. It calls a `@Injectable()` service which talks to Firebase
6. Returned value is auto-serialized to JSON and sent back

Database: Firebase Realtime Database

A hosted NoSQL JSON-tree database. Pros:

- Zero setup (no installing/running a DB server)
- Free tier suitable for a personal app
- Simple read/write API

Cons:

- No SQL joins — you fetch lists and filter in code
- Lacks proper indexes — fine for small datasets, slow at scale
- Not relational — easy to end up with inconsistent data if you're not careful

For this app's scale (one user, a few hundred records), the tradeoff is fine.

Authentication: JWT (JSON Web Tokens)

When a user logs in, the backend gives them a **token** (a long string). The frontend stores it and sends it with every future request. The backend verifies the token to know which user is calling.

A JWT looks like:

```
eyJhbGciOiJIUzI1NiIs...header.eyJzdWIiOiJ1c2VyMTIz...payload.signature
```

It's three base64-encoded parts: header, payload (contains user id + expiry), and a signature. The backend can verify the signature with a secret key (`JWT_SECRET` in `.env`) and trust the payload without a database lookup.

This app uses **two** tokens:

- **Access token** — short-lived (15 min), sent on every request
- **Refresh token** — long-lived (7 days), used only to get a new access token

When the access token expires, the frontend silently calls `/api/auth/refresh` with the refresh token to get a new access token. If that fails too (refresh token also expired), the user is logged out.

5. Project structure (annotated)

Frontend (`frontend/src/app/`)

```

app/
├─ app.config.ts           ← bootstrap config: router + http + interceptors
├─ app.routes.ts          ← all URL → component mappings (lazy-loaded)
├─ app.component.ts       ← root component (just <router-outlet>)
├─
├─ core/                  ← cross-cutting concerns (loaded once, app-wide)
│   ├─ guards/
│   │   ├─ auth.guard.ts   ← blocks unauth users from /dashboard etc.
│   │   └─ no-auth.guard.ts ← blocks logged-in users from /auth/login
│   ├─ interceptors/
│   │   └─ auth.interceptor.ts ← attaches "Bearer <token>" to outgoing requests,
│   │                                   silently refreshes on 401, redirects to login on failure
│   ├─ models/             ← TypeScript interfaces (just type definitions)
│   │   ├─ user.model.ts
│   │   ├─ task.model.ts
│   │   ├─ goal.model.ts
│   │   ├─ habit.model.ts
│   │   └─ category.model.ts
│   └─ services/           ← singleton classes injectable everywhere
│       ├─ auth.service.ts   ← login/register/refresh, current user signal
│       ├─ token-storage.service.ts ← localStorage wrapper for the tokens
│       ├─ theme.service.ts   ← dark/light mode toggle
│       ├─ toast.service.ts   ← in-app notification system
│       └─ google-calendar.service.ts
│
├─ shared/                 ← reusable UI components used across features
│   └─ components/
│       ├─ layout/          ← the app shell: sidebar + topbar + <router-outlet>
│       ├─ sidebar/         ← left nav, theme toggle, user block
│       ├─ topbar/          ← optional top bar (not currently rendered)
│       ├─ modal/           ← dialog wrapper used by all forms
│       ├─ confirm-dialog/   ← "Are you sure?" reusable popup
│       ├─ toast-container/  ← renders toasts from ToastService
│       └─ category-manager/ ← category CRUD popup
│
└─ features/               ← one folder per page/feature
    ├─ auth/login | register/
    ├─ dashboard/           ← "Dashboard" page
    ├─ tasks/
    │   ├─ task.service.ts   ← talks to /api/tasks
    │   ├─ task-list/        ← "My Plans" page
    │   ├─ task-form/        ← form rendered inside the modal
    │   └─ task-detail/      ← /tasks/:id detail page with timer
    ├─ goals/
    │   ├─ goal.service.ts
    │   └─ goal-list/

```

```

|   |   └─ goal-form/
|   |   └─ goal-detail/      ← milestones + mini-goals UI
|   └─ habits/
|       └─ habit.service.ts
|       └─ habit-list/      ← "Daily Routine" page
|       └─ habit-form/
└─ calendar/                ← monthly grid
└─ categories/              ← category service only (UI is in shared/)
└─ profile/                 ← edit profile + change password

```

Each feature folder is **self-contained**: component + service + (optional) form. The service inside each feature is what talks to the backend.

Backend (backend/src/)

```

src/
└─ main.ts                  ← server entry: creates app, enables CORS, listens on port 3000
└─ app.module.ts            ← root module, imports every feature module
|
└─ auth/                   ← register/login/refresh/logout
    └─ auth.module.ts       ← bundles controller + service + JWT strategies
    └─ auth.controller.ts   ← HTTP endpoints (@Post('register'), etc.)
    └─ auth.service.ts      ← business logic: hash password, generate tokens
    └─ strategies/
        └─ jwt.strategy.ts   ← validates access tokens on every request
        └─ jwt-refresh.strategy.ts ← validates refresh tokens on /auth/refresh
    └─ guards/              ← JwtAuthGuard (global), JwtRefreshGuard
    └─ dto/                 ← request body shapes (RegisterDto, LoginDto)
|
└─ users/                  ← /api/users/profile GET/PATCH
└─ tasks/                  ← /api/tasks/* CRUD + timer endpoints
└─ goals/                  ← /api/goals/* + nested milestones + mini-goals
└─ habits/                 ← /api/habits/* + check-ins, streaks, heatmap
└─ categories/             ← /api/categories/* CRUD
└─ dashboard/              ← /api/dashboard/stats|activity|calendar (aggregations)
└─ google-calendar/        ← /api/google/* OAuth + sync
|
└─ common/                 ← shared utilities
    └─ decorators/
        └─ public.decorator.ts ← @Public() — skip JWT guard for register/login/refresh
        └─ current-user.decorator.ts ← @CurrentUser('id') — extract user from JWT
|
└─ prisma/                 ← name is historical; this is the Firebase wrapper
    └─ prisma.module.ts     ← exposes FirebaseService globally
    └─ firebase.service.ts  ← .get(), .set(), .update(), .remove(), .getList()

```

Every feature module follows the same skeleton:

- A `XxxModule` (just wires things up)
- A `XxxController` (HTTP routing — small file)
- A `XxxService` (business logic + Firebase reads/writes — the meat)
- `dto/` folder with classes describing what request bodies should look like

6. Following a click: full data flow example

Let's trace "User checks off the Wakeup habit on the dashboard" through every layer.

Step 1 — User clicks the checkbox

In the browser, the user sees this in [dashboard.component.ts](#):

```
<button class="th-row" (click)="toggleHabit(h)"> ... </button>
```

The `(click)` binding tells Angular: when this button is clicked, call `toggleHabit(h)` on the component class.

Step 2 — Component delegates to the service

`toggleHabit()` in the dashboard component:

```
toggleHabit(h: Habit) {  
  this.habitService.toggleToday(h.id).subscribe({ ... });  
}
```

It hands off to `HabitService` — a singleton injected via `inject(HabitService)`. The service is shared by both the dashboard and the habits page; that's how the BehaviorSubject keeps them in sync.

Step 3 — Service does an optimistic update, then HTTP

[habit.service.ts](#):

```
toggleToday(id: string) {  
  const original = this.habits$.value.find((h) => h.id === id);  
  if (original) {  
    // Flip UI state instantly, before waiting for the server.  
    this.replaceHabit(this.optimisticToggleToday(original));  
  }  
  return this.http.post<Habit>(  
    `${this.base}/${id}/checkin?today=${this.localTodayKey()}`,  
    {}  
  ).pipe(  
    tap((u) => this.replaceHabit(u)), // replace with server's authoritative version  
    catchError((err) => {  
      if (original) this.replaceHabit(original); // revert on failure  
      return throwError(() => err);  
    })  
  );  
}
```

Key things happening here:

- **Optimistic update:** the local in-memory `habits$` BehaviorSubject is flipped immediately so the UI updates the moment you click — no waiting for the server.
- **HTTP POST:** a request is sent to `http://localhost:3000/api/habits/<id>/checkin?today=2026-05-23`.
- **Replace:** when the server responds, the local state is replaced with the real one (in case the server computed something the optimistic update couldn't, like the streak).
- **Revert on error:** if the network fails, the UI flips back.

Step 4 — HTTP interceptor adds the auth token

Before the request actually leaves the browser, [auth.interceptor.ts](#) sees it:

```
const token = tokens.getAccessToken();  
const authReq = token
```

```

    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;
return next(authReq).pipe(...);

```

So the request that actually goes out has a header:

```
Authorization: Bearer eyJhbGc...
```

The interceptor will also catch 401 responses and try to refresh the token automatically — see [Authentication explained](#).

Step 5 — Browser sends the request, NestJS receives it

The request lands on the NestJS server. Order of processing:

1. **CORS middleware** — checks the request is allowed from `http://localhost:4200`. See `app.enableCors(...)` in `main.ts`.
2. **Global JWT guard** (`JwtAuthGuard` registered in `app.module.ts`) — verifies the `Authorization` header. It uses Passport JWT strategy to decode the token and load the user. If invalid, returns 401.
3. **Routing** — NestJS finds the controller method matching `POST /api/habits/:id/checkin`. That's [habits.controller.ts](#):

```

@Post('/:id/checkin')
checkinToday(
  @CurrentUser('id') userId: string,
  @Param('id') id: string,
  @Query('today') today?: string,
) {
  return this.habitsService.toggleCheckin(userId, id, today, today);
}

```

4. **Decorators inject params:** `@CurrentUser('id')` extracts the user id from the JWT-validated request; `@Param('id')` reads the URL segment; `@Query('today')` reads the query string.
5. **Delegate to service:** hands off to `HabitsService.toggleCheckin(...)`.

Step 6 — Service reads/writes Firebase

[habits.service.ts](#):

```

async toggleCheckin(userId, habitId, date, today) {
  await this.ensureOwnership(userId, habitId); // 403 if user doesn't own this habit
  const todayKey = this.resolveToday(today);
  const targetDate = date ?? todayKey;

  const checkins = await this.firebase.getList<...>('habitCheckins');
  const existing = checkins.find((c) =>
    c.habitId === habitId && c.date === targetDate && c.userId === userId
  );

  if (existing) {
    await this.firebase.remove(`habitCheckins/${existing.id}`); // un-check
  } else {
    const id = randomUUID();
    await this.firebase.ref(`habitCheckins/${id}`).set({
      id, habitId, userId, date: targetDate, completedAt: new Date().toISOString(),
    });
  }
}

```

```
// Re-fetch habit + checkins, compute streak/scheduled/history, return enriched object
const habit = await this.firebase.get(`habits/${habitId}`);
const fresh = (await this.firebase.getList(...)).filter((c) => c.userId === userId);
return this.enrich(habit, fresh, todayKey);
}
```

`this.firebase` is the `FirebaseService` — its `.ref().set()` writes a JSON node into the Firebase Realtime Database. `getList('habitCheckins')` reads all check-ins (across all users) and we filter by `userId` in code.

Step 7 — Firebase persists the change

Firebase stores the JSON tree update on its servers. From now on, this data exists forever (until deleted) and is visible to any future request from this user.

Step 8 — Response travels back

The service returns the enriched habit object (habit + computed `streak`, `doneToday`, `scheduledToday`, `history[30]`). NestJS serializes it to JSON and sends it back over HTTP:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": "abc-123",
  "title": "Wakeup",
  "doneToday": true,
  "streak": 1,
  ...
}
```

Step 9 — Frontend service receives the response

Back in `habit.service.ts`, the `.tap((u) => this.replaceHabit(u))` runs. The local `habits$` `BehaviorSubject` emits the new value. Every component subscribed to it (the dashboard and the habits page) re-renders with the updated streak / done state.

Step 10 — User sees the result

In about 200–500 ms (most of which was the network roundtrip), the UI shows:

- Checkbox filled
- Streak number bumped from 0 → 1
- Toast: "Nice — 1-day streak"

But thanks to the optimistic update in step 3, the checkbox actually flipped **instantly** (within ~16ms) when the click happened — the server response just confirmed it.

7. Authentication explained

Why JWT?

We need a way for the backend to know **which user** is sending a request. Options:

1. **Sessions** (server stores user-id ↔ session-id in memory/DB; browser sends a cookie). Simple but stateful — every server in a cluster needs access to the session store.
2. **JWTs** (the user's id is encoded into a signed token; backend verifies the signature). Stateless — any server can verify it with the secret key.

This app uses JWTs.

Two-token system

Token	Lifespan	Used for	Stored in
Access token	15 minutes	Every API call	localStorage (in browser)
Refresh token	7 days	Only for /auth/refresh	localStorage (in browser) + hashed in DB

Short-lived access tokens limit damage if one leaks. Refresh tokens are stored hashed on the backend so they can be revoked (logout invalidates them server-side).

Login flow

1. User submits email + password on `/auth/login`.
2. Frontend calls `POST /api/auth/login` with `{email, password}`.
3. Backend (`auth.service.ts`):
 - Looks up the user by email in Firebase
 - Compares the password with `bcrypt` against the stored hash
 - Generates two JWTs (access + refresh) signed with `JWT_SECRET` / `JWT_REFRESH_SECRET`
 - Stores the **hashed** refresh token in Firebase
 - Returns `{ user, accessToken, refreshToken }`
4. Frontend stores both tokens in `localStorage` and sets `currentUser` signal.
5. Router navigates to `/dashboard`.

Auto-attach token on every request

The HTTP interceptor (see step 4 above) automatically reads the access token from `localStorage` and adds it as a header on every outgoing request. You don't have to remember to do this in each service.

Auto-refresh when access token expires

The interceptor also catches 401 responses:

```
catchError((err: HttpErrorResponse) => {
  if (err.status === 401 && !isRefreshing && !req.url.includes('/auth/')) {
    isRefreshing = true;
    return auth.refresh().pipe(
      switchMap((res) => { // refresh succeeded
        isRefreshing = false;
        const retried = req.clone({ setHeaders: { Authorization: `Bearer ${res.accessToken}` } });
        return next(retried); // retry the original request
      }),
      catchError((refreshErr) => { // refresh failed
        isRefreshing = false;
        auth.clearSession(); // clear tokens, redirect to /auth/login
        return throwError(() => refreshErr);
      })
    );
  }
  return throwError(() => err);
});
```

So if your access token expires while you're using the app, the user never notices — the next request gets a fresh access token transparently.

If even the refresh token is expired (you've been gone for a week), `auth.clearSession()` synchronously clears `localStorage`, resets the user signal, and navigates to `/auth/login`.

Route guards

`AuthGuard` checks `isLoggedIn()` (just "is there a token in localStorage?") before allowing access to `/dashboard`, `/tasks`, etc. If not, redirect to `/auth/login`.

`noAuthGuard` does the opposite for `/auth/*`: if you're already logged in, send you to `/dashboard`.

The `@Public()` decorator

The backend's `JwtAuthGuard` is registered globally — every endpoint requires a token by default. Endpoints like `/auth/register` and `/auth/login` mark themselves `@Public()` to opt out:

```
@Public()
@Post('login')
login(@Body() dto: LoginDto) { ... }
```

The guard checks for this decorator and skips the JWT check.

8. Feature deep-dives

8.1 My Plans (Tasks)

A "plan" is the generic term — it can be a task, trip, train trip, dinner reservation, meeting, event, or reminder. They all share one schema with a `type` field.

Backend ([backend/src/tasks/](#)):

- `TasksService.findAll(userId, filters)` — fetches tasks from Firebase, filters by `userId`, optionally by `status/priority/type`
- `create/update/remove/findOne` — standard CRUD
- `startTimer/stopTimer` — for the built-in time tracker on the detail page

Frontend:

- `TaskService` (`features/tasks/task.service.ts`) holds a `tasks$ BehaviorSubject`. Components subscribe to it.
- `TaskListComponent` ("My Plans" page) shows the list with:
 - Type filter chips (All / Task / Trip / Journey / Food / ...)
 - Status + priority dropdowns
 - Pagination**: 15 per page, computed locally from `tasks$`
 - DONE-last sorting**: completed tasks sink to the bottom, sorted by date desc among themselves
 - Edit/delete buttons + inline status dropdown
- `TaskFormComponent` (rendered inside `ModalComponent`) handles create/edit
- `TaskDetailComponent` (`/tasks/:id`) shows full info + a working timer

8.2 Goals

A **goal** has many **milestones**, each milestone has many **mini-goals**. Progress is auto-calculated from completed milestones.

Backend data shape:

```
goals/<id>      { title, userId, progress, ... }
milestones/<id> { goalId, status, order, ... }
minigoals/<id>  { milestoneId, goalId, status, ... }
```

This is a "flat" relational model on top of Firebase — milestones reference their parent goal by ID; we don't nest them in the JSON tree (so we can query milestones globally without loading every goal).

`GoalsService.findOne()` does the join in code: fetches the goal, fetches its milestones (filter by `goalId`), and for each milestone fetches its mini-goals (filter by `milestoneId`). For 1 user with <100 goals this is fast; at scale you'd want indexes.

Progress computation: every time a milestone is created/completed/deleted, the service recomputes `goal.progress = completed / total * 100` and writes it back.

Frontend:

- `GoalListComponent` — card per goal with progress bar
- `GoalDetailComponent` — accordion of milestones, each with mini-goal checkboxes
- Reorderable milestones via PATCH `/:goalId/milestones/reorder`
- Resources field on each goal (free-text array)

8.3 Daily Routine (Habits)

The most data-heavy module — let's go through it carefully because it has streaks, heatmaps, timezones, and backfilling.

Schema

```
habits/<id> {
  id, title, description,
  weekdays: [0..6],          // 0=Sun..6=Sat — which days the habit is scheduled
  startTime, endTime,       // "HH:MM" 24h — optional time window
  reminderEnabled: boolean,
  icon, color,
  archived: boolean,
  userId, createdAt, updatedAt
}

habitCheckins/<id> {
  id, habitId, userId,
  date: "YYYY-MM-DD",       // one check-in per (habit, date)
  completedAt: ISO timestamp
}
```

A check-in is an atomic record: "user X completed habit Y on date Z". Toggling on/off creates/removes one of these. The streak and heatmap are **computed at read time** by walking through check-ins — they're never stored as fields.

Streak computation

Walk backwards from "today":

- If the day is scheduled (its weekday is in `habit.weekdays`):
 - If a check-in exists for that date → `streak++`
 - If no check-in AND it's today → don't break (user hasn't checked in yet today)
 - If no check-in AND it's a past day → break (streak ends here)
- Stop scanning once cursor < `habit.createdAt` (or after 366 days as a safety cap)

This gives the **current** streak, which is the number of consecutive scheduled days completed up to today.

30-day heatmap

For each of the last 30 days, compute `{date, done, scheduled}`. The frontend renders these as small color-coded cells:

- Dim gray = not scheduled
- Faint red = scheduled but missed
- Green gradient = done (intensity from light to solid based on aggregate completion %)

Days **before** the habit was created are marked `scheduled: false` so they don't appear as "missed" (the user wasn't tracking yet).

Timezones — the tricky part

Firebase doesn't know about timezones. The backend originally used `new Date().toISOString().split('T')[0]` for "today" — but that's the **UTC** date. A user in India (UTC+5:30) at 12:01 AM Saturday would have the server still thinking it's Friday → habits scheduled for weekdays would appear scheduled "today" even on a Saturday morning.

Fix: the frontend sends `?today=YYYY-MM-DD` (computed from the user's local clock) on every habit API call. The backend uses that as the reference for streak/heatmap/scheduling. Check-ins are stored under whatever date the client says it is.

See `localTodayKey()` in `habit.service.ts` and `resolveToday()` in `habits.service.ts` (backend).

Backfilling past dates

The Daily Routine page has a date selector (← / → arrows) that lets you navigate back up to 29 days. When viewing a past date:

- The checklist shows habits scheduled for that day's weekday
- The check state comes from `habit.history[]` (the date's `done` cell)
- Clicking a checkbox calls `toggleDate(habitId, selectedDate)` instead of `toggleToday`
- Streak recomputes server-side; client doesn't try to predict it for past-day changes

Optimistic updates

The service flips local state synchronously before the HTTP call returns (see step 3 in the data flow walkthrough). Streak math is exact for "today" toggles (+1 or -1). For past-date toggles the streak isn't touched until the server responds (could ripple in non-obvious ways).

8.4 Calendar

A monthly grid view. The dashboard service has `/api/dashboard/calendar?year=2026&month=5` which returns every plan with a `dueDate` overlapping that month.

The frontend's `CalendarComponent` renders a 6×7 grid (Sun-Sat columns). Each day cell shows:

- Date number
- Office-hours overlay if it's a weekday (and not a holiday/leave)
- Colored chips for each plan, color-coded by `type` (PLAN_TYPES constant)
- Click a date → opens a modal with that day's full list + an "Add Plan" form

Holiday and leave toggles are stored locally per date.

8.5 Dashboard

The most aggregated page. It loads in parallel:

- `GET /api/dashboard/stats` — total tasks / completed today / active goals
- `TaskService.getToday()` — today's scheduled tasks
- `GoalService.loadAll()` — first 4 active goals for the progress section
- `HabitService.loadAll()` — all habits (for the Daily Routine widget)

The Daily Routine widget on the dashboard renders:

- **Progress ring** (SVG circle with `stroke-dasharray` animation) showing today's % completed
- **Three pill stats:** 30-day consistency %, best current streak, total habit count
- **Interactive checklist** — same UX as the habits page (click to toggle)
- **30-day aggregate heatmap** — for each day, the cell color shows what % of scheduled habits got done that day. Legend bar shows the gradient.

When you toggle a habit from the dashboard, the `HabitService.habits$` `BehaviorSubject` emits → both the dashboard widget and the habits page (if open in another tab) sync instantly because they both subscribe.

9. Recurring patterns you'll see in the code

Standalone Angular components

Every component declares `standalone: true` and lists its own imports:

```
@Component({
  selector: 'app-habit-list',
  standalone: true,
  imports: [ModalComponent, HabitFormComponent, ...],
  template: `...`,
  styles: [...],
})
export class HabitListComponent { ... }
```

No NgModule needed — you just import the component class wherever you use it.

Signals + computed properties

Modern Angular reactive primitives:

```
habits = signal<Habit[]>([]); // mutable state
todayHabits = computed(() => // derived state, auto-updates when habits() changes
  this.habits().filter(h => h.scheduledToday)
);

// Usage in template: {{ todayHabits().length }}
// Usage in code: this.habits.set([...new array]);
```

BehaviorSubject for cross-component state

```
@Injectable({ providedIn: 'root' }) // singleton
export class HabitService {
  habits$ = new BehaviorSubject<Habit[]>([]);

  loadAll() {
    return this.http.get<Habit[]>(...).pipe(
      tap((h) => this.habits$.next(h)) // emit new value to all subscribers
    );
  }
}

// In component:
this.habitService.habits$.subscribe((habits) => this.habits.set(habits));
```

Any component that subscribes to `habits$` will see updates when any other component (or the service itself) calls `.next(...)`. This is how the dashboard and the habits page stay in sync.

HttpClient + observables

All HTTP calls return RxJS `Observable<T>` :

```
this.http.get<Habit[]>('/api/habits').subscribe({
  next: (habits) => { /* success */ },
  error: (err) => { /* failure */ },
});
```

You can chain operators with `.pipe()` :

- `tap((x) => ...)` — side effect, doesn't change the value
- `map((x) => transform(x))` — transform the value
- `catchError((err) => ...)` — handle errors, return a fallback or rethrow

Reactive Forms

For any form:

```
form = this.fb.group({
  title: ['', Validators.required],
  description: [''],
});

// In template:
<form [formGroup]="form" (ngSubmit)="submit()">
  <input formControlName="title" />
  <input formControlName="description" />
  <button type="submit" [disabled]="form.invalid">Save</button>
</form>
```

`form.value` is `{title, description}` ; `form.valid / invalid` reflects validator state.

NestJS controllers + services + DTOs

The same shape repeats for every backend feature:

Controller — just routes, no logic:

```
@Controller('habits')
export class HabitsController {
  constructor(private habitsService: HabitsService) {}
  @Get() findAll(@CurrentUser('id') userId: string) {
    return this.habitsService.findAll(userId);
  }
}
```

Service — actual business logic + Firebase reads/writes:

```
@Injectable()
export class HabitsService {
  constructor(private firebase: FirebaseService) {}
  async findAll(userId: string) {
    const habits = await this.firebase.getList<Habit>('habits');
    return habits.filter((h) => h.userId === userId);
  }
}
```

DTO — incoming body validation:

```
export class CreateHabitDto {
  @IsString() @MaxLength(120) title: string;
  @IsOptional() @IsArray() weekdays?: number[];
```

```
...
}
```

The global `ValidationPipe` automatically validates `@Body()` `dto: CreateHabitDto` against these decorators and returns 400 with a useful error if it fails.

Optimistic updates

When a click should feel instant but needs a server roundtrip:

1. Flip the local state immediately
2. Fire the HTTP call
3. On response: replace local state with the server's authoritative version
4. On error: revert the local state

See `HabitService.toggleToday()` for the pattern.

10. Adding a new feature module

Suppose you want to add a "Notes" feature.

Backend

1. **Create the folder:** `backend/src/notes/`
2. **DTO** — `dto/create-note.dto.ts` :

```
import { IsString, IsOptional, MaxLength } from 'class-validator';
export class CreateNoteDto {
  @IsString() @MaxLength(200) title: string;
  @IsOptional() @IsString() body?: string;
}
```

3. **Service** — `notes.service.ts` :

```
@Injectable()
export class NotesService {
  constructor(private firebase: FirebaseService) {}
  async findAll(userId: string) {
    return (await this.firebase.getList<any>('notes')).filter(n => n.userId === userId);
  }
  async create(userId: string, dto: CreateNoteDto) {
    const id = randomUUID();
    const note = { id, ...dto, userId, createdAt: new Date().toISOString() };
    await this.firebase.ref(`notes/${id}`).set(note);
    return note;
  }
}
```

4. **Controller** — `notes.controller.ts` :

```
@Controller('notes')
export class NotesController {
  constructor(private service: NotesService) {}
  @Get() findAll(@CurrentUser('id') userId: string) { return this.service.findAll(userId); }
  @Post() create(@CurrentUser('id') userId: string, @Body() dto: CreateNoteDto) {
    return this.service.create(userId, dto);
  }
}
```

```
}  
}
```

5. **Module** — `notes.module.ts` :

```
@Module({ controllers: [NotesController], providers: [NotesService] })  
export class NotesModule {}
```

6. **Wire into** `app.module.ts` : add `NotesModule` to the `imports` array.

That's it — backend is done. Test with curl: `curl http://localhost:3000/api/notes -H "Authorization: Bearer ..."` .

Frontend

1. **Model** — `frontend/src/app/core/models/note.model.ts` :

```
export interface Note { id: string; title: string; body: string | null; userId: string; createdAt:  
string; }
```

2. **Service** — `frontend/src/app/features/notes/note.service.ts` :

```
@Injectable({ providedIn: 'root' })  
export class NoteService {  
  private http = inject(HttpClient);  
  notes$ = new BehaviorSubject<Note[]>([]);  
  loadAll() { return this.http.get<Note[]>(`${environment.apiUrl}/notes`).pipe(tap(n =>  
this.notes$.next(n))); }  
  create(dto: Partial<Note>) { return this.http.post<Note>(`${environment.apiUrl}/notes`,  
dto).pipe(tap(n => this.notes$.next([...this.notes$.value, n]))); }  
}
```

3. **List component** — render the notes from `noteService.notes$ | async` .

4. **Add a route** in `app.routes.ts` :

```
{  
  path: 'notes',  
  loadComponent: () => import('./features/notes/note-list/note-list.component').then(m =>  
m.NoteListComponent),  
}
```

5. **Add a sidebar link** in `sidebar.component.ts` .

Done. Roughly mirror the structure of `habits/` or `goals/` and you'll be consistent with the rest of the codebase.

11. Running the project locally

Prerequisites

- Node.js 18 or higher (`node --version`)
- A Firebase project with Realtime Database enabled
- A Firebase service account key file

Backend setup

1. `cd backend && npm install`

- Put your Firebase service account JSON at `backend/serviceAccountKey.json` (download from Firebase Console → Project Settings → Service Accounts → "Generate new private key")
- Create `backend/.env` :

```
JWT_SECRET="any-random-32-char-string"
JWT_REFRESH_SECRET="a-different-random-32-char-string"
FIREBASE_DATABASE_URL="https://YOUR-PROJECT-default-rtdb.firebaseio.com"
```

- `npm run start:dev` — starts the backend with hot reload on `http://localhost:3000/api`

Frontend setup

- `cd frontend && npm install`
- `npm start` — runs `ng serve` and opens `http://localhost:4200`

Common issues

Symptom	Fix
EADDRINUSE: address already in use :::3000	A previous backend process didn't die. Find the PID: <code>netstat -ano grep :3000</code> then kill it.
Login fails immediately	Backend isn't running, or <code>JWT_SECRET</code> changed since the user was created (tokens issued before become invalid).
Habits show on wrong day	Make sure the frontend is sending <code>?today=...</code> — see <code>HabitService.todayParam()</code> .
Frontend can't reach backend	CORS issue. Backend must list <code>http://localhost:4200</code> in <code>app.enableCors({origin: [...]}).</code>
Firebase connection error on backend start	<code>serviceAccountKey.json</code> missing OR <code>FIREBASE_DATABASE_URL</code> WRONG.

12. Glossary

- API:** A list of URLs (endpoints) the backend exposes for the frontend to call.
- BehaviorSubject:** An RxJS class that holds a current value and re-emits it to new subscribers. Used as a local cache for entity lists.
- CORS:** Cross-Origin Resource Sharing. A browser security rule that blocks frontend code on one domain from calling APIs on another unless the API allows it. The backend's `enableCors()` whitelists the frontend's origin.
- DTO** (Data Transfer Object): A TypeScript class describing what shape a request body should have. Used by NestJS for validation.
- Decorator:** A `@Something()` annotation in front of a class or method. Both Angular and NestJS use them heavily (`@Component`, `@Injectable`, `@Controller`, `@Post`, etc.).
- Dependency injection:** A pattern where a class declares what it needs in its constructor (or via `inject()`) and the framework provides instances. Lets you write `private auth = inject(AuthService)` instead of `new AuthService()`.
- Endpoint:** A specific URL + HTTP verb the backend handles, e.g. `GET /api/habits/today`.
- Guard** (NestJS): A class that runs before a controller method to allow or block the request (e.g., check JWT).
- Guard** (Angular Router): A function that runs before navigating to a route to allow or block it (e.g., redirect to login if not authenticated).
- HTTP interceptor:** A function that wraps every outgoing HTTP request, often to add headers or catch errors globally.
- JSON:** Plain-text format for structured data: `{"key": "value", "n": 42}`.
- JWT** (JSON Web Token): A signed text token that encodes user identity. Sent as `Authorization: Bearer <token>` on every request.
- Middleware:** Code that runs between request arrival and the controller handler (e.g., body parsing, CORS, auth).
- Module** (NestJS): A class that bundles related controllers + services + imports.
- Observable** (RxJS): A stream of values over time. `.subscribe(callback)` to react to each emission. HTTP calls return Observables.

- **Optimistic update:** Update the UI immediately on user action, before the server confirms. Revert on error.
- **Reactive Forms:** Angular's class-based form model with validators (`FormGroup` , `FormControl`).
- **REST API:** A style of HTTP API where URLs represent resources and verbs (GET/POST/PATCH/DELETE) represent operations. This project follows REST loosely.
- **Service** (Angular): An `@Injectable()` class for sharing logic/state across components.
- **Service** (NestJS): An `@Injectable()` class for business logic, separated from controllers.
- **Signal** (Angular): A reactive primitive `signal(initial)` with `.set()` / `.update()` and reactive `.computed()` .
- **Standalone component:** An Angular component that declares its own imports and doesn't need an `NgModule`.
- **State management:** How an app keeps its UI in sync with data. This project uses signals for component-local state and `BehaviorSubjects` in services for shared state.
- `tap()` : An RxJS operator that lets you do a side effect (like updating a `BehaviorSubject`) without changing the stream's value.

If something in the code isn't clear, search the file for the relevant decorator or service name — most things in this codebase follow a couple of repeating patterns, so once you understand one feature module, the others will read fast.